

CSCI1103 · PROGRAMMING PRINCIPLES I

02

Variables and Data Types

Teaching the program to decide and to repeat — logical operators, conditionals, and the three loop forms that drive every algorithm from here on.

KBTU · School of IT and Engineering · Fall 2026

LEARNING OBJECTIVES

Learning objectives

By the end of this lecture, you will be able to:

- 01 Evaluate logical expressions using AND, OR, XOR and NOT.
- 02 Write comparisons and combine them into if / nested-if decisions.
- 03 Use pow, round, ceil and floor correctly.
- 04 Distinguish a single char from a string and pick the right one for the job.
- 05 Choose between for, while and do-while, and use continue/break to control a loop's flow.

LOGIC

Logical operators

- ◆ `&&` (AND) — true only if both sides are true.
- ◆ `||` (OR) — true if at least one side is true.
- ◆ `^` (XOR) — true only if the sides differ.
- ◆ `!` (NOT) — flips a bool.

COMBINING CONDITIONS

```
bool isAdult = age >= 18;  
bool hasTicket = true;  
bool canEnter = isAdult && hasTicket;  
bool eitherDay = (day == "Sat") || (day == "Sun");
```

WORTH REMEMBERING

You won't reach for XOR often as a beginner — think of it as “exactly one of two switches is on.” It pays off later, when lecture 3 gets to bitmasks.

COMPARISONS

Logical comparisons

- ◆ Relational operators: `== != < > <= >=` — every comparison evaluates to a `bool`.
- ◆ Common bug: `=` assigns, `==` compares — `if (x = 5)` compiles but is (almost) always wrong.
- ◆ Comparisons chain through logical operators: `if (score >= 0 && score <= 100)`

DECISIONS

Conditional statements

- ◆ `if (condition) { ... }` runs a block only when the condition is true.
- ◆ `else` runs when it's false; `else if` chains further conditions.
- ◆ Nested if: an `if` inside another `if` — useful, but readability drops fast past two levels.

GRADING WITH NESTED IF

```
if (score >= 90) {  
    grade = 'A';  
} else if (score >= 75) {  
    grade = 'B';  
} else {  
    if (attended) grade = 'C';  
    else grade = 'F';  
}
```

MATH LIBRARY, CONTINUED

pow, round, ceil, floor

- ◆ `pow(x, y)` → x^y , returns a `double` even for whole numbers.
- ◆ `round(x)` → nearest whole number (as a double).
- ◆ `ceil(x)` → rounds up; `floor(x)` → rounds down.
- ◆ All four need `#include <cmath>`.

```
round(4.5); // 5  
ceil(4.1);  // 5  
floor(4.9); // 4
```

TEXT DATA

Introduction to char and string

- ◆ `char` holds exactly one character in single quotes: `'A'`.
- ◆ `string` holds a sequence of characters in double quotes: `"Almaty"` — needs `#include <string>`.
- ◆ A string can be indexed like an array: `name[0]` gives its first char.
- ◆ Concatenate strings with `+`; you cannot concatenate two chars the same way.

REPETITION

Loop operators

- ◆ `for (int i = 0; i < n; i++) { ... }` — best when the number of repetitions is known up front.
- ◆ `while (condition) { ... }` — checks before each pass; may run zero times.
- ◆ `do { ... } while (condition);` — checks after each pass; always runs at least once.

THE SAME COUNT-UP, THREE WAYS

```
for (int i = 1; i <= 5; i++) cout << i;
```

```
int i = 1;  
while (i <= 5) { cout << i; i++; }
```

```
int j = 1;  
do { cout << j; j++; } while (j <= 5);
```


CONTROLLING A LOOP

continue and break

- ◆ `break;` exits the loop immediately, skipping any remaining iterations.
- ◆ `continue;` skips only the rest of the current iteration and moves to the next one.
- ◆ Both work inside `for`, `while` and `do-while`.

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue; // skip evens  
    if (i > 7) break;         // stop early  
    cout << i << " ";  
}
```

EASY TO MIX UP

`continue` does not exit the loop like `break` does — it only skips to the next iteration. Trace the example on the right line by line if that's not obvious yet.

RECAP

Key takeaways

- ✓ Logical operators (`&&` `||` `^` `!`) combine boolean conditions; comparisons always produce a `bool`.
- ✓ `if` / `else if` / `else` and nested `if` select which block of code runs.
- ✓ `pow`, `round`, `ceil`, `floor` round out the math toolkit from lecture 1.
- ✓ `for`, `while` and `do-while` cover every repetition pattern; `continue` and `break` fine-tune them.

STUDIO TIME

Laboratory work #2

Practice conditionals and all three loop types on small decision-making and counting problems.

- ◆ Write a grading program using nested if / else if.
- ◆ Rewrite the same counting task with for, while, and do-while to compare them.
- ◆ Use continue/break to filter values inside a loop.

Quiz 1 (lectures 1-2) is coming in week 4 – this lab is the direct rehearsal for it.

UP NEXT

Next: Numeric Systems, Bits and Bitwise Operators

Lecture 3 goes under the hood of the `int` and `char` types just introduced — binary and hexadecimal, and the bitwise operators that manipulate individual bits.

L3 – Numeric Systems, Bits and Bitwise Operators · Week 3